INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

UNIDAD DE APRENDIZAJE: BASES DE DATOS

PRÁCTICA 1: MODELO ENTIDAD-RELACIÓN

Ejercicios 1 y 2
Control de versiones y contenedores

Profesor: Hurtado Avilés Gabriel

Integrantes

Damas Garnica Adrian

Esparza Ramón Fernando Adán

Ramos Padrón Jesus Emmanuel

Hernandez Barrios Samuel Rodrigo

Tercer semestre

Grupo: 3CV4

Fecha: 18 de septiembre de 2026

Ejercicios 1 y 2 — Investigación: Control de Versiones y Contenedores

Ejercicio 1. Control de versiones con Git y GitHub

1. ¿Qué es un sistema de control de versiones?

Un sistema de control de versiones (SCV) es una herramienta de software que registra los cambios realizados sobre un conjunto de archivos a lo largo del tiempo, permitiendo recuperar cualquier versión anterior, comparar diferencias entre versiones y saber quién modificó qué y cuándo. En un trabajo en equipo, el problema concreto que resuelve es la coordinación del trabajo simultáneo sobre los mismos archivos: sin un SCV, dos personas que editan el mismo archivo terminan sobrescribiendo el trabajo de la otra, o el equipo recurre a soluciones artesanales como enviarse archivos por correo con nombres como "proyecto_final_v2_bueno.docx", lo que genera pérdida de trabajo, ambigüedad sobre cuál es la versión vigente y ausencia de un historial confiable. Git, en particular, resuelve esto de forma distribuida: cada colaborador tiene una copia completa del historial del proyecto en su propia máquina, por lo que puede trabajar sin conexión y fusionar sus cambios con los del resto del equipo cuando lo necesite.

2. Diferencia entre Git y GitHub

Git y GitHub no son sinónimos, aunque suelen confundirse. Git es el sistema de control de versiones en sí: un programa que se ejecuta localmente en la computadora del desarrollador y que se encarga de rastrear los cambios, crear ramas, fusionar código y mantener el historial de confirmaciones. Git puede usarse por completo sin conexión a internet y sin depender de ningún servicio externo. GitHub, en cambio, es una plataforma en la nube que aloja repositorios de Git y añade funcionalidad de colaboración alrededor de ellos: control de acceso, Pull Requests, revisión de código, seguimiento de incidencias (issues), integración continua y una interfaz web para explorar el historial. En otras palabras, Git es la herramienta; GitHub es un servicio que hospeda proyectos que usan esa herramienta y facilita que varias personas colaboren sobre ellos. Existen alternativas a GitHub que también hospedan repositorios de Git, como GitLab o Bitbucket, lo cual confirma que Git y su plataforma de hospedaje son conceptos independientes.

3. Definiciones con ejemplo

- Repositorio: es la carpeta del proyecto junto con todo su historial de cambios, gestionada por Git. Ejemplo: la carpeta practica1-bd, una vez inicializada con git init, se convierte en un repositorio que contiene tanto los archivos actuales como cada versión anterior de ellos.
- Confirmación (commit): es una fotografía del estado del proyecto en un momento dado, junto con un mensaje que describe qué cambió y por qué. Ejemplo: después de agregar el archivo compose.yaml, se ejecuta git commit -m "Agrega configuración de PostgreSQL en Docker", lo que crea un punto de restauración identificado con un hash único.
- Rama (branch): es una línea de desarrollo independiente que parte de un punto del historial, usada para trabajar en una funcionalidad sin afectar el código estable. Ejemplo:

se crea la rama feature/diagrama-er para trabajar el modelo entidad-relación mientras la rama main permanece intacta.

- Fusión (merge): es la operación que integra los cambios de una rama dentro de otra. Ejemplo: al terminar el trabajo en feature/diagrama-er, se fusiona esa rama dentro de main para incorporar el diagrama al proyecto principal.
- Conflicto de fusión: ocurre cuando Git no puede combinar automáticamente los cambios de dos ramas porque ambas modificaron las mismas líneas de un archivo, y requiere que una persona decida manualmente qué versión conservar. Ejemplo: dos integrantes del equipo modifican el mismo párrafo del README en ramas distintas; al fusionar, Git marca el conflicto con los marcadores <<<<<<<, ======= y >>>>>>> y detiene la fusión hasta que se resuelva.
- Pull Request (PR): es una solicitud formal, dentro de GitHub, para incorporar los cambios de una rama a otra, que además sirve como espacio de revisión de código antes de aceptarlos. Ejemplo: al terminar la rama feature/diagrama-er, se abre un PR hacia main, describiendo qué se agregó, para que un compañero lo revise antes de fusionarlo.
- Archivo .gitignore: es un archivo de texto que indica a Git qué archivos o carpetas debe ignorar y no rastrear dentro del repositorio. Ejemplo: en este proyecto, .gitignore excluye archivos temporales del sistema operativo y carpetas de configuración local que no deben compartirse con el equipo.
- Archivo README.md: es el documento de presentación del repositorio, escrito en Markdown, que describe de qué trata el proyecto, cómo instalarlo o usarlo, y quién lo elaboró. Ejemplo: el README.md de este repositorio incluye el nombre del autor, grupo, carrera y un índice con enlaces a cada entregable.

4. Flujo de trabajo basado en ramas y revisión entre pares

Un flujo de trabajo basado en ramas consiste en mantener la rama principal (main) siempre estable y funcional, mientras que cualquier cambio nuevo —una función, una corrección, un documento— se desarrolla en una rama separada creada específicamente para esa tarea. Una vez que el trabajo en esa rama está terminado, se abre un Pull Request para integrarlo a main, en lugar de subir los cambios directamente. Este flujo es la base de metodologías más elaboradas como Git Flow o GitHub Flow, y su propósito es aislar el trabajo en progreso del código que ya funciona, evitando que un cambio a medias rompa lo que el resto del equipo está usando.

El código se revisa entre pares antes de fusionarse porque una segunda persona que no escribió el código detecta con más facilidad errores lógicos, malas prácticas, código duplicado o decisiones de diseño discutibles que el autor original, inmerso en el problema, puede pasar por alto. Además, la revisión entre pares distribuye el conocimiento del proyecto entre todo el equipo —evitando que solo una persona entienda cierta parte del sistema— y funciona como un control de calidad antes de que el código llegue a la rama estable, reduciendo la probabilidad de introducir errores en producción.

Ejercicio 2. El sistema gestor en un contenedor: Docker

1. Contenedor frente a máquina virtual

Un contenedor es una unidad de software que empaqueta una aplicación junto con sus dependencias (bibliotecas, variables de entorno, configuración) para que se ejecute de forma consistente en cualquier entorno, compartiendo el kernel del sistema operativo anfitrión en lugar de virtualizar hardware completo. Esto lo diferencia de una máquina virtual (VM) en tres aspectos concretos: en arranque, un contenedor inicia en segundos porque no necesita cargar un sistema operativo completo, mientras que una VM tarda minutos al arrancar un kernel propio; en tamaño, una imagen de contenedor suele pesar decenas o cientos de megabytes al reutilizar el kernel del host, mientras que una VM pesa gigabytes porque incluye un sistema operativo entero; y en aislamiento, la VM ofrece aislamiento más fuerte al virtualizar hardware y ejecutar un kernel independiente, mientras que el contenedor aísla procesos a nivel de sistema operativo (usando namespaces y cgroups en Linux), lo cual es más ligero pero comparte el mismo kernel con el host y con otros contenedores.

2. Definiciones

- Imagen: es una plantilla inmutable de solo lectura que contiene el sistema de archivos, las dependencias y la configuración necesarias para ejecutar una aplicación. Ejemplo: la imagen postgres:16 contiene el binario de PostgreSQL 16 y todo lo necesario para correrlo.
- Contenedor: es una instancia en ejecución de una imagen; es el proceso vivo que corre a partir de esa plantilla. Ejemplo: al ejecutar docker run postgres:16 se crea el contenedor pg-practica1, un proceso aislado corriendo esa imagen.
- Volumen: es un mecanismo de almacenamiento persistente gestionado por Docker que existe fuera del ciclo de vida del contenedor, de modo que los datos sobreviven aunque el contenedor se elimine. Ejemplo: el volumen pgdata almacena los archivos de datos de PostgreSQL de forma independiente al contenedor que los usa.
- Puerto publicado: es el mapeo de un puerto interno del contenedor hacia un puerto del sistema anfitrión, para que servicios externos puedan conectarse. Ejemplo: la bandera -p 5432:5432 publica el puerto 5432 del contenedor de PostgreSQL en el puerto 5432 de la máquina host.

3. Por qué el volumen es indispensable

El volumen es indispensable en un sistema gestor de bases de datos porque, por diseño, el sistema de archivos de un contenedor es efímero: pertenece al ciclo de vida de ese contenedor específico. Si el contenedor se detiene y se elimina (docker rm), su capa de escritura desaparece junto con él. Si no se declara un volumen, todo lo que la base de datos haya almacenado —tablas, registros, índices— se pierde de forma irreversible en el momento en que se elimina el contenedor, y volver a levantar uno nuevo a partir de la misma imagen arrancará una base de datos completamente vacía. Declarar un volumen desacopla los datos del contenedor: el contenedor puede destruirse y recrearse cuantas veces sea necesario (por ejemplo, para actualizar la versión de PostgreSQL) sin que eso afecte la información, siempre que se vuelva a montar el mismo volumen.